

不是模型不行而是 context 不行

最近我在重新审视一个问题：为什么同样的模型，不同人/团队用出来的结果差距那么大？但这显然不是新问题。

最近看 Debois 3 月 18 日在 QCon London 的演讲《Context Is the New Code》，让我对这个问题有了更清晰的认知。其实并不是 Debois 提出了完全没想到的新东西，而是他把我隐约感觉到的东西用更听得懂的语言说清楚了。

一、我最近有一个粗糙的判断，不是模型不行是 context 不行

根本原因是其不可见性，代码写错通常会报错，系统会主动告诉你出了问题，但 context 写错，agent 不会报错，它只是默默地输出不想要的结果，但最危险的是我们可能以为那就是模型能力不足，根本不会去查 context。尤其最近看到很多人说公司模型不好，这种“不可见性”的感觉更加强烈，所以抛开 context 谈模型能力都是可耻的！这种就是“沉默失败”的危险，让 context 的错误极难被发现，也极难被定位。就像 Debois 把现在的 context 管理状态叫做 Cowboy coding——AGENTS.md 从其他地方复制粘贴、prompt 手动编辑、没有版本记录、没有人知道哪条规则还在生效、哪条已经过时，我觉得这个描述非常准确，但他没有说透的是 Cowboy coding 的代码至少会崩，但 context 只会漂移、会慢慢偏离你的意图，而察觉不到。

二、我发现我们团队里有大量知识从来没有被记录下来过……

“见某个客户用某个方法和客户谈判会导致失败，所以不能这么谈”这是某个或多个 BD 踩过坑之后形成的判断，活在他脑子里或是大家的共识，从来没有变成正式文档被记录下来，这可能是几百次物理世界信息 review 慢慢磨出来的记忆。这些知识在物理世界里通过老人带新人、会议里讨论、谈判里吸收，逐渐形成有效群体意识。但 agent 没有这种社会化过程，它每次启动都是全新的，缺乏记忆，没有被纠正然后学到了的能力，它的经验完全等于你在 context 里告诉它的东西。那些你觉得大家都懂，不用写的知识，对 agent 来说是完全不存在的。所以隐性知识的断层，在物理世界里是个慢性问题——新人多踩几次坑就学会了。但在有 agent 参与的团队里，这个问题会被无限放大：agent 永远在踩同样的坑，因为它没有记忆。

Debois 说 AI 给了我们一个前所未有的强烈理由去把隐性知识显性化——不这么做，agent 会一直在那里出错，而我们永远不知道根源在哪里。我完全认同这个判断，并且认为这是团队引入 AI agent 之后最先应该解决的工程问题，没有之一。所以最近，我开始要求团队充分利用好小美听写“凡事要听写”，试图把物理世界的信息最大程度地补充 Agent 的 context，用最简单但相对笨的办法完成最基础的 context 工程。

三、context 的成熟度决定了 agent 能力的上限

我最近在 AI 的帮助下，尝试用一个“context 成熟度阶梯”来评估团队的 context 工程水平，框架也是来自 Debois，但每一级的理解有不同的诠释。

即兴指令：每次对话临时写提示，没有沉淀。Agent 的能力完全取决于你当次的描述质量，不可复现，不可扩展。

规则文件：把常用指令写成 AGENTS.md，自动注入。这是大多数团队今天的位置，也是上边说的 Cowboy coding 主要形态。

结构化 skill：把规则文件进一步封装成 skill 包，包含脚本、文档、工具调用。context 从文本变成可执行的工程资产。

动态信息集成：通过各种工具实时从学城、会议、讨论拉取 context，而不只是依赖静态文件。context 从静态变成动态、可被感知。

规格驱动开发：prompt 本身写成规格说明，由 agent 分解为子任务执行，context 变成一种设计语言，我理解接近于 harness。

我感觉这个阶梯里第一二级和第三四级和第五级之间存在两层本质变化的鸿沟。第一、二级的 Context 是静态死板的：它不会随任务变化，无法感知代码库的实际状态，也学不会团队的最新决策。到了第三、四级，系统性开始显现，Context 变成了可测试、可复用、可持续演进的资产。而到了第五级，本质上是 Context 层面的高级框架应用、不再指令 Agent“怎么做，让它自己按照已有的框架规则、规划路径。但是很难。

四、context 需要像代码一样被测试

这是我觉得 Debois 框架里最重要的观察。Debois 设计了三层 context 测试体系：linter 级格式验证、LLM as judge 的语义理解验证、以及最核心的单元测试级 eval。eval 的具体案例是这样的：团队规定所有 API 路径必须以 /awesome 为前缀，把这条规范写进 context，然后用一个 eval 测试 agent 生成的 API 路径是否包含 /awesome/user。如果 eval 失败，调试对象是 context 不是模型。

我们以前用 agent 的思维模式是 agent 输出不对→模型理解能力有问题→换模型或者改 prompt，但 eval 驱动的思维模式是 agent 输出不对→context 规范表述有歧义→修改 context 规范→重跑 eval→验证修复效果，但 context 不够，这个又跑不起来……感觉这是在间接测试一个 agent 的控制系统，我们不能直接控制 agent 的输出，但可以控制它接收的 context，然后通过观察输出来推断 context 质量。这有点像通过可控 input 来影响 output 的做功方式，把这个认知转变到使用 agent 的 context 上。当开始用 eval 来测试 context，每一次 eval 失败，都是一条你没有写清楚的规范，告诉你 context 的哪个部分需要被补充。有了足够的 context 的积累，我认为这是团队在使用 agent 上最应该建立的习惯“context 管理”，方法很笨、成本极低，但收益极高。

今天很多团队在讨论用哪个模型，我认为现在很多模型的能力都比一个智力普通的人强。其实这个问题的战略价值在持续下降，因为模型是公共基建“你方唱罢我登场”差距在缩小、切换成本更是在降低。但团队积累的 context：业务的、讨论的、踩坑的，是私有的、是在日积月累的迭代中沉淀出来的，没有办法被换一个更好的模型来替代。这个甚至和亚马逊飞轮

有异曲同工之处、也可以用这个这套方式来尝试 AI 组织业务变革，即更多项目/任务→更多 context (input) →更明确的规范 (PDCA) →更好的 agent 输出 (output) →更多项目/任务。两个飞轮差别在于，亚马逊业务飞轮需要规模才能转起来，而 Context 飞轮从第一天就可以开始积累就能开始转。

所以，不是模型不行而是 context 不行，这是很笨的方法但长期有价值，现在就开始积累 context 资产，随便模型怎么换 context 永流传。